universidade de aveiro
deti
departamento de eletrónica,
telecomunicações e informática

Distributed Systems
Summary

Eurico Pedrosa <efp@ua.pt>

2^{nd} semester - 2025′26

Introduction and Foundations

Core Foundations

- **Definition**: A collection of autonomous computers interconnected by a network, appearing to users as a single coherent system.
- **Key Characteristics**: Components have no shared memory, no global clock, and are often highly dynamic.
- **Distribution Layers**: Middleware resides between applications and operating systems to provide transparency.

Major Design Goals

- **Resource Sharing**: Enabling efficient remote access to hardware, data, and services while managing concurrency and access control.
- **Openness**: Systems that easily integrate heterogeneous components through standard interfaces and decoupled policies.
- **Scalability**: The ability to handle growth in users (size), distance (geography), or organizations (administration) without major redesign.
- **Dependability**: Providing reliable, available, and safe services even when partial failures occur.

Distribution Transparency Types

- **Access**: Hides differences in data representation and access methods.
- **Location**: Hides where an object is physically located.
- **Replication**: Hides the fact that multiple copies of an object exist.
- **Failure**: Hides the failure and recovery of an object from the user.
- **Migration & Relocation**: Hides that objects may move during use or between sessions.

System Classifications

- **High-Performance Computing**: Focuses on raw processing power through clusters (homogeneous) or grids (federated/heterogeneous).
- **Information Systems**: Focused on data integration and transaction processing (e.g., Enterprise Application Integration).
- **Pervasive Systems**: Small, mobile, and context-aware devices (sensors, smartphones) blended into the environment.

Distributed Transactions and ACID

- **Atomicity**: Operations are “all or nothing”; if one part fails, the whole transaction rolls back.
- **Consistency**: Transactions must transition the system from one valid state to another, preserving invariants.
- **Isolation**: Concurrent transactions do not interfere with each other.
- **Durability**: Once committed, data remains stored even in the event of system failures.

Common Pitfalls (False Assumptions)

- The network is reliable and secure.
- Latency is zero and bandwidth is infinite.
- The topology is stable and there is only one administrator.
- Transport cost is zero and the network is homogeneous.

Processes, Threads and Concurrency

Fundamental Concepts

- **Process**: A program in execution managed by the Operating System, providing isolation through private address spaces and contexts.
- **Thread**: A single flow of control within a process that shares the same address space but maintains its own processor context, such as the stack and program counter.
- **Isolation vs. Performance**: Processes offer high protection but expensive context switches; threads offer low-overhead concurrency but require synchronization for shared data.

Multi-threading Models

- **User-Level Threads**: Managed by a library without kernel knowledge; fast to switch but can block the entire process on I/O.
- **Kernel-Level Threads**: Managed by the OS; can utilize multiple CPUs and handle blocking I/O, but context switching is slower than user-level.
- **Hybrid Models**: Combining user and kernel threads to balance performance and flexibility.

Server Architectures

- **Multithreading**: Uses a thread pool to handle multiple requests in parallel, suitable for blocking I/O.
- **Single-Threaded**: Processes requests sequentially; simple to implement but results in poor performance during blocking operations.
- **Event-Driven (Finite State Machine)**: Uses non-blocking I/O and event loops to handle many concurrent connections with a single thread.

Concurrency and Synchronization

- **Race Condition**: Occurs when multiple threads access and modify shared data concurrently, leading to unpredictable results.
- **Critical Section**: A segment of code where shared resources are accessed and must be protected.
- **Mutual Exclusion**: Ensures that only one thread can execute in a critical section at any given time.

Deadlocks

- **Definition**: A state where a set of processes are blocked because each is waiting for a resource held by another process in the set.
- **Necessary Conditions**: Mutual exclusion, hold and wait, no preemption, and circular wait.
- **Prevention Strategies**: Imposing a strict linear ordering on resource acquisition or requiring all resources to be requested at once.

Architectures

Fundamental Concepts

- **Software vs. System Architecture**: Software architecture refers to the logical organization of software components and their interactions, while system architecture refers to the physical placement of those components on machines.
- **Middleware Layer**: Positioned between the application and the operating system, it provides distribution transparency by masking network complexity from users.
- **Decoupling**: Modern architectures aim to separate functional components to improve scalability, maintainability, and fault tolerance.

Centralized Architectures

- **Client-Server Model**: A traditional model where a central server manages functionality and responds to requests from remote clients.
- **Multi-tier Architectures**:
  - **2-tier**: Includes thin clients (only UI on client) or fat clients (processing logic on client).
  - **3-tier**: Separates the user interface, processing logic, and data management into three distinct layers.

Decentralized Architectures

- **Peer-to-Peer (P2P)**: Nodes act as both clients and servers, distributing the workload without a central coordinator.
- **Structured P2P**: Uses deterministic routing, typically based on Distributed Hash Tables (DHTs) like Chord, to locate resources.
- **Unstructured P2P**: Relies on randomized algorithms such as flooding or random walks to find data.
- **Hierarchical P2P**: Combines centralized and decentralized elements by using “super-peers” to manage clusters of regular nodes.

Hybrid and Specialized Models

- **Content Delivery Networks (CDNs)**: Distribute content by placing replicas on edge servers closer to users to reduce latency.
- **Cloud Computing**: Provides on-demand resources through virtualization, categorized as IaaS, PaaS, or SaaS.
- **Edge and Fog Computing**: Moves computation closer to the data source (IoT devices) to minimize response times and bandwidth usage.

Blockchain Architectures

- **Distributed Ledger**: A decentralized database where transactions are cryptographically linked in blocks and validated by participants.
- **Public (Permissionless)**: Open to anyone; uses heavy consensus mechanisms like Proof of Work.
- **Private (Permissioned)**: Restricted to verified participants; uses lighter and faster consensus protocols.
- **Key Benefits**: High transparency and security due to the lack of a single point of failure.
- **Challenges**: High storage requirements for replication and slower global consensus compared to centralized systems.

Communication

Fundamental Concepts

- **Definition**: The mechanism allowing processes on different machines to exchange information.
- **Protocols**: Sets of rules governing the format, contents, and meaning of messages.
- **Layering**: Dividing communication tasks into independent levels to ensure modularity.

Layered Protocols and the OSI Model

- **OSI Reference Model**: A 7-layer model where each layer provides services to the layer above it.
- **Key Layers**:
  - **Application Layer**: Contains protocols for specific applications (e.g., HTTP).
  - **Transport Layer**: Handles end-to-end communication, ensuring data arrives correctly (e.g., TCP, UDP).
  - **Network Layer**: Responsible for routing messages through the network (e.g., IP).

Communication Classifications

- **Persistence**:
  - **Persistent**: Messages are stored by the middleware until they can be delivered.
  - **Transient**: Messages are discarded if they cannot be delivered immediately.
- **Synchronicity**:
  - **Asynchronous**: The sender continues immediately after sending a message.
  - **Synchronous**: The sender blocks until the message is received or processed.

Remote Procedure Call (RPC)

- **Goal**: To make a remote service call appear as if it were a local function call.
- **Access Transparency**: Achieved by hiding the details of network communication.
- **Mechanism**:
  - **Client/Server Stubs**: Handle the packing (marshalling) and unpacking (unmarshalling) of parameters.
  - **IDL (Interface Definition Language)**: Used to specify the interface independently of the programming language.

Message-Oriented Middleware (MOM)

- **Function**: Supports high-level asynchronous communication through message queuing.
- **Loose Coupling**: Decouples senders and receivers in both time and space.
- **Persistence**: Messages are stored in queues, ensuring delivery even if the receiver is temporarily offline.

Large-Scale Dissemination

- **Gossip/Epidemic Protocols**: Used for fast and reliable data spreading in large-scale systems.
- **Mechanism**: Nodes periodically exchange information with random peers, mimicking the spread of a virus.
- **Benefits**: Highly scalable and resilient to node failures.

Coordination

Fundamental Concepts

- **Definition**: Coordination allows processes to cooperate and synchronize, ensuring they act as a single, coherent system.
- **Objectives**: Agreement on the relative ordering of events and ensuring mutual exclusion for shared resources.
- **Relationship to Synchronization**: Coordination encapsulates process synchronization (waiting for others) and data synchronization (consistency).

Clock Synchronization: Physical Clocks

- **The Problem**: Computers have internal physical clocks that drift over time, leading to different times on different machines.
- **Network Time Protocol (NTP)**: A hierarchical system (strata) used to synchronize clocks over a network.
- **Reference Broadcast Synchronization (RBS)**: Synchronization based on the arrival time of a broadcast message rather than the sender's time.

Clock Synchronization: Logical Clocks

- **Lamport Timestamps**: Uses the “happens-before” relation ($a \rightarrow b$) to establish a partial ordering of events.
- **Vector Clocks**: An extension of Lamport clocks that captures causality and allows for the detection of concurrent events.

Distributed Mutual Exclusion

- **Purpose**: Ensuring that only one process can access a shared resource (critical section) at a time.
- **Algorithms**:
  - **Centralized**: A single coordinator manages requests and grants permissions.
  - **Distributed**: Based on timestamps and multicasting to all processes to gain permission.
  - **Token Ring**: Processes are organized in a logical ring, and a token is passed around; only the holder can access the resource.

Leader Election

- **Definition**: The process of selecting a single process to act as a coordinator or leader.
- **Bully Algorithm**: The process with the highest ID “bullies” others to become the leader after a failure is detected.
- **Ring-Based Election**: Processes pass election messages around a logical ring to identify the highest ID.

Location Systems (Network Coordinates)

- **Centralized (Landmarks)**: Landmark nodes measure latencies to assign coordinates to other nodes.
- **Decentralized (Vivaldi)**: Models the network as a system of physical springs where nodes adjust coordinates based on latency “forces” to minimize local error.
- **Adaptive Step (δ)**: Used in Vivaldi to ensure convergence by adjusting the size of coordinate movements.

Consistency and Replication

Fundamental Concepts

- **Replication**: The maintenance of multiple copies of data across different nodes.
- **Primary Goals**:
  - **Reliability**: Ensuring system operation continues even if some replicas fail.
  - **Performance**: Improving scalability and reducing latency by placing data closer to users.
- **Consistency Challenge**: Ensuring that updates to one copy are propagated so all replicas remain synchronized.

Consistency Models

- **Data-Centric Consistency**: Focuses on the internal state of the data store.
- **Sequential**: All processes see the same order of operations.
- **Linearizability**: Operations appear instantaneous and globally ordered.
- **Eventual Consistency**: If no updates occur for a long time, all replicas will eventually converge to the same state.
- **Client-Centric Consistency**: Focuses on what an individual client perceives.
- **Monotonic Reads**: If a process reads a value, subsequent reads return that value or a more recent one.
- **Monotonic Writes**: Writes by the same process are processed in the order they were issued.
- **Read Your Writes**: A process always sees the effects of its own previous writes.

Replica Management

- **Replica Placement**: Deciding where and when to create replicas (Permanent, Server-initiated, or Client-initiated).
- **Update Propagation**:
  - **State vs. Operations**: Passing the updated data itself or the command to perform the update.
  - **Push vs. Pull**: Servers pushing updates to replicas vs. replicas pulling updates when needed.

Consistency Protocols

- **Primary-Based Protocols**: A single “primary” node coordinates all writes to ensure a fixed order.
  - **Remote-Write**: Updates are sent to a fixed primary node.
  - **Local-Write**: The primary status moves to the node currently performing the write.
- **Quorum-Based Protocols**: A majority of replicas must agree before an operation is finalized.
  - **Condition**: $N_R + N_W > N$ (where N_R is read quorum, N_W is write quorum, and N is total replicas).
  - **Effect**: Guarantees that any read quorum contains at least one replica from the most recent write.

Fault Tolerance

Fundamental Concepts

- **Dependability**: The core requirement for fault-tolerant systems, reflecting the degree to which a system can be relied upon to operate as expected.
- **Partial Failure**: A characteristic of distributed systems where one component fails while others remain operational.
- **Goal**: To mask failures and the subsequent recovery from the end user or application.
- **Strategies**: Building dependable systems involves preventing, tolerating, removing, and forecasting faults.

Dependability Metrics

- **Availability**: The system is ready to provide services at any given time.
- **Reliability**: The ability of a system to run continuously without failure over a period of time.
- **Safety**: Ensuring that if a system fails, it does so without causing catastrophic consequences.
- **Maintainability**: How easily a failed system can be repaired.

Failure Models

- **Crash Failure**: A server halts but was working correctly until the halt.
- **Omission Failure**: A server fails to respond to incoming requests or fail to send a response.
- **Timing Failure**: A server's response lies outside the specified time interval.
- **Response Failure**: The server's response is simply incorrect (Value or State transition failure).
- **Arbitrary (Byzantine) Failure**: A server may produce arbitrary responses at arbitrary times.

Failure Masking by Redundancy

- **Information Redundancy**: Adding extra bits to allow for error detection or correction.
- **Time Redundancy**: Performing an action again if it fails (e.g., retrying a transaction).
- **Physical Redundancy**: Adding extra equipment or processes to the system.
- **K-fault tolerant**: A system can survive k concurrent faults.
  - **Silent Failures**: Requires $k+1$ replicas to mask k faults.
  - **Byzantine Failures**: Requires $2k+1$ (voting) or $3k+1$ (agreement) replicas to mask k faults.

Consensus and Agreement

- **Objective**: Getting all non-faulty processes to reach a common decision.
- **Paxos**: A foundational protocol for reaching consensus in a network of unreliable processors.
- **Raft**: A consensus algorithm designed to be easier to understand and implement than Paxos, focusing on leader election and log replication.

Recovery Mechanisms

- **Backward Recovery**: Bringing the system from its current erroneous state back into a previously correct state (Check-pointing).
- **Forward Recovery**: Moving the system into a new state from which it can continue to operate correctly.
- **Message Logging**: Recording messages to replay them during recovery to avoid “orphans” (processes dependent on lost messages).
  - **Pessimistic**: Blocks until the message is logged; preferred in practice for simplicity.
  - **Optimistic**: Allows execution to continue and handles dependencies via rollbacks if a crash occurs.

Resources

Resources

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.